Vergleich mit TensorFlow

Teil 3 — dieselben Netze mit einer Bibliothek

Inhaltsverzeichnis

Aufbau des Vergleichs 1
XOR 1
7-Segment-Decoder 2
Was der Vergleich zeigt 2
Selbst ausprobieren 3

Teil 3 der Angabe trainiert dieselben Netze mit TensorFlow und vergleicht Genauigkeit, Trainingsgeschwindigkeit und Konvergenz mit der eigenen Implementierung. Das Programm dazu liegt im Ordner `tensorflow/` des Repos.

Aufbau des Vergleichs

Das Keras-Modell hat dieselbe Form wie das eigene Netz: Dense-Layer mit Sigmoid, gerechnet in `float64`. Auch das Training ist gleich: stochastischer Gradientenabstieg mit einem Update pro Sample in fester Reihenfolge, $\alpha = 0,1$, 20 000 Epochen und die Kostenfunktion $J = \frac{1}{2}\sum(\hat{y} - y)^2$ aus der Theorie. Der 7-Segment-Decoder hat wie in Teil 2 16 Hidden-Neuronen; die Angabe nennt für Teil 3 „z. B. 10″.

`keras.losses.MeanSquaredError` mittelt über die Outputs statt zu halbieren. Beim 7-Segment-Decoder wären die Gradienten damit um den Faktor $\frac{2}{7}$ kleiner — das entspräche einer anderen Lernrate, und der Vergleich wäre nicht mehr fair.

Jedes Problem wird dreimal trainiert:

Lauf	Startgewichte	Zweck
eigene Implementierung	gleichverteilt in $[-1, 1)$, Seed 0 bzw. 42	Referenz
TensorFlow, gleicher Start	genau die Startgewichte der eigenen Implementierung	prüft die eigene Backpropagation
TensorFlow, Keras-Init	Glorot-uniform, Biases 0, Seed 0 bzw. 42	übliche Verwendung von TensorFlow

XOR

Netz 2-2-1.

Lauf	Endkosten	richtig	Kosten < 0,01 ab	Zeit
eigene Implementierung	0,000486	4/4	Epoche 6 003	1,3 s
TensorFlow, gleicher Start	0,000486	4/4	Epoche 6 003	2,3 s

Lauf	Endkosten	richtig	Kosten < 0,01 ab	Zeit
TensorFlow, Keras-Init	0,000479	4/4	Epoche 5 151	2,2 s

Epoche	eigene	TF, gleicher Start	TF, Keras-Init
0	0,131458	0,131458	0,132694
2 000	0,125323	0,125323	0,112603
4 000	0,099136	0,099136	0,043017
6 000	0,010023	0,010023	0,005407
10 000	0,001667	0,001667	0,001477
20 000	0,000486	0,000486	0,000479

Input	Soll	eigene	TF, gleicher Start	TF, Keras-Init
[0, 0]	0	0,0343	0,0343	0,0277
[0, 1]	1	0,9699	0,9699	0,9677
[1, 0]	1	0,9708	0,9708	0,9677
[1, 1]	0	0,0308	0,0308	0,0313

7-Segment-Decoder

Netz 4-16-7.

Lauf	Endkosten	richtig	Kosten < 0,01 ab	Zeit
eigene Implementierung	0,000486	10/10	Epoche 2 298	15,2 s
TensorFlow, gleicher Start	0,000486	10/10	Epoche 2 298	5,1 s
TensorFlow, Keras-Init	0,000435	10/10	Epoche 2 329	5,1 s

Epoche	eigene	TF, gleicher Start	TF, Keras-Init
0	1,325987	1,325987	0,862386
2 000	0,012936	0,012936	0,013691
4 000	0,004012	0,004012	0,003822
10 000	0,001136	0,001136	0,001032
20 000	0,000486	0,000486	0,000435

Die größte Abweichung einer Ausgabe vom Sollwert beträgt 0,030 (eigene und TF mit gleichem Start) bzw. 0,024 (Keras-Init).

Was der Vergleich zeigt

- **Genauigkeit:** Alle drei Läufe lösen beide Probleme vollständig, die Endkosten liegen nahe beieinander.

- **Gleicher Start, gleicher Weg:** Mit denselben Startgewichten stimmen die Kostenverläufe auf sechs Stellen überein, die Ausgaben weichen nach 20 000 Epochen um höchstens $4 \cdot 10^{-10}$ voneinander ab. TensorFlow berechnet die Gradienten per automatischer Differenzierung, also völlig unabhängig von der Herleitung in der Theorie. Das bestätigt die eigene Backpropagation über das ganze Training, zusätzlich zum numerischen Gradienten-Check. Der kleine Rest entsteht, weil beide in anderer Reihenfolge runden.
- **Konvergenz:** Mit der Keras-Initialisierung verlässt das XOR-Netz das Plateau früher (Kosten unter 0,01 ab Epoche 5 151 statt 6 003); beim 7-Segment-Decoder sind beide fast gleich schnell. Das hängt hier an den Startgewichten, nicht an der Bibliothek.
- **Geschwindigkeit:** TensorFlow rechnet jeden Layer als Matrixoperation in einem kompilierten Graphen. Beim 7-Segment-Decoder ist es damit dreimal so schnell wie die eigene Python-Version, die für jedes Neuron einzeln numpy aufruft. Beim kleinen XOR-Netz überwiegt der Aufbau des Graphen (rund eine Sekunde), dort ist die eigene Version schneller. Die C#-Version braucht für dieselben Trainings 0,05 s bzw. 1,1 s (Ergebnisse): Bei so kleinen Netzen zählt der Aufwand pro Schritt mehr als die Rechenleistung.

Alle Zeiten wurden auf demselben PC gemessen; die TensorFlow-Zeiten enthalten den Aufbau des Graphen.

Selbst ausprobieren

TensorFlow 2.21 gibt es nicht für Python 3.14, der Ordner hat deshalb eine eigene Umgebung mit Python 3.12 (Einrichtung im `README.md` des Repos). Im Ordner `tensorflow/`:

```
.venv\Scripts\python -m tf_comparison
.venv\Scripts\python -m tf_comparison xor --epochs 5000
.venv\Scripts\python -m pytest
```

Die Tests prüfen, dass kopierte Gewichte dieselben Vorhersagen liefern, dass ein einzelner SGD-Schritt in TensorFlow genau das Update der eigenen Implementierung ergibt und dass beide über 500 Epochen denselben Verlauf nehmen.